PPT Outline Intelligent Generation and Content Completion System Review Meeting Summary

1. Main Content of the Meeting

1.1. Project Positioning and Core Architecture

The review meeting first reviewed the project's starting point and core positioning: Addressing the common pain points of AI PPT tools on the market, such as "template stuffing" and empty content, the team clearly defined its R&D positioning as "an intelligent PPT creation assistance system for in-depth content generation." In terms of product design, the system constructs a highly flexible dual-path creation mode: For users with mature materials (such as research reports, survey data, or meeting minutes), the system automatically parses the document and quickly extracts the outline skeleton through a "report-driven mode," directly bypassing the requirements collection stage; while for users with only vague theme ideas, a "guided creation mode" conducts multiple rounds of dialogue, systematically collecting five core dimensions: theme, audience, duration, style, and key points, deeply concretizing the user's vague intentions. To ensure the stability of large model generation, the project team adhered to a rigorous "four-level outline JSON Schema specification" and designed deterministic engineering rules as a hard safety net (such as mandatory interception for topics without a theme, gradient-based questioning based on missing fields, and a reserved escape route for one-click generation). This not only effectively avoided structural deviations caused by model illusions at key decision points but also laid a solid logical foundation for downstream content enrichment and visualization rendering.

1.2. Technology Implementation and Core Module Implementation

In terms of technology implementation and architecture implementation, the project has successfully migrated from the system blueprint to the front-end and back-end infrastructure. The system is built on the FastAPI asynchronous framework and React front-end in an integrated manner, with a clearly defined five-layer architecture (API, Common, Infrastructure, Workers, Modules) to ensure clear responsibilities and dependencies. In terms of foundation design, the team innovatively used Python-side callback functions to automatically infer table names and audit timestamps, perfectly adapting to the ORM serialization mechanism in the asynchronous SQLAlchemy environment and completely eliminating the system crash risks caused by implicit lazy loading in asynchronous contexts. The meeting highlighted the integration of the system's core engine—the RAG retrieval module—with the asynchronous queue. This module not only handles vector indexing and similarity retrieval but also addresses the engineering pain point of formula and image parsing distortion identified in the previous review. It introduces the Marker

document parsing framework to convert PDFs into Markdown with LaTeX formula annotations. Simultaneously, it utilizes a multimodal model for OCR and semantic descriptive transformation of charts, deeply integrating them into the vector database retrieval. Furthermore, in conjunction with the Redis Stream asynchronous task queue, the system employs a rigorous execution flow of "committing DB transactions first, then enqueuing XADD," completely eliminating the potential for "orphan tasks" caused by excessively fast worker consumption. A session-level concurrency protection lock is introduced at the entry point, ensuring data idempotency and session security in high-concurrency scenarios while maintaining information density and retrieval accuracy.

1.3. Project Progress and Future Plans

As of June 1st, the overall project progress is highly aligned with the Gantt chart plan, and all planned tasks prior to June 1st have been completed to a high standard. During this sprint period, the team accumulated approximately 60 hours of effective work, achieving 100% completion of the core system framework. They successfully launched the backend infrastructure, ensured the frontend engineering environment was ready, closed-looped frontend-backend communication, and completed testing of key modules. The development is currently steadily transitioning from core functionality development to quality assessment, system integration, and delivery preparation. The final sprint phase (June 1st to June 14th) is expected to involve 32 hours of work. The focus will shift to system construction evaluation and multi-model horizontal comparison. By writing ROUGE-L and BERTScore evaluation scripts, we will conduct horizontal comparisons of three objective data sources: "pure LLM, LLM+RAG, and LLM+RAG+DeepResearch," and introduce an LLM-as-a-Judge mechanism. At the same time, the team will launch a "human double-blind review test" with 3-5 people. Under anonymized conditions, we will quantitatively score the outline sample and annotate it with illusory text. We will combine subjective and objective indicators to draw an evaluation radar chart, conduct quantitative verification of the system, and simultaneously complete defect repair and Docker containerized prototype deployment to ensure high-quality final delivery of the system on June 14th.

2. Comments and Advice of the Reviewers

During the Milestone 2 review session, the evaluation panel provided constructive feedback and professional guidance to steer the project toward successful final delivery. The polished and translated review comments are detailed below:

1. Huang Yicheng (黄毅成)

The system adopts a cutting-edge technological stack by selecting the FastAPI full-asynchronous framework paired with React + TypeScript, and effectively introduces LangGraph to handle complex RAG pipelines and multi-turn search orchestration. This architecture strongly aligns with

contemporary large language model application trends. However, because the system stacks multiple latency-heavy paths—including Server-Sent Events (SSE) streaming, the DeepResearch network search API, and RAG vector database queries—it is highly susceptible to severe fluctuations in response time. Given that the current report evaluates performance primarily through qualitative descriptions, it is highly recommended that the team supplement their documentation with concrete performance stress-testing data and establish definitive high-concurrency service degradation contingency plans in the next phase.

2. Xu Junda (许君达)

The project features a highly logical and clear system pathway design. Bifurcating the PPT generation journey into "Report-Driven" and "Guided Creation" scenarios, and anchoring them with structured outlines, RAG retrieval, and DeepResearch workflows, demonstrates a comprehensive consideration of diverse user entry points. For the upcoming evaluation phase, the team should distinctly differentiate the evaluation benchmarks between these two modes: the Report-Driven mode should place heavier emphasis on the absolute accuracy of text summarization, whereas the Guided Creation mode should prioritize the efficacy of requirement gathering and the value of external information expansion. Relying solely on a uniform generation quality score may obscure the system's actual strengths and behavioral bottlenecks under different operational contexts.

3. Zuo Lingxu (左凌旭)

The system exhibits complete functional coverage, backed by an exceptionally robust project management framework. The outline generation speed is remarkably swift, and the inclusion of a source tracing mechanism provides great transparency. On the client side, the frontend interface layout effectively highlights key actions, ensuring intuitive and simplified document editing. To enhance the business case of the system, it is recommended to introduce an API token consumption analysis to calculate the average financial cost incurred per outline generation. Furthermore, the team should actively proceed with multi-model comparative benchmarking tests to enrich their empirical findings.

4. Chen Yiming (陈奕名)

This project is highly polished, shows rapid development progress, and currently delivers excellent core functionality. The Work Breakdown Structure (WBS) and Risk Breakdown Structure (RBS) frameworks are meticulously detailed and granular. Regarding the comparative data against human baselines presented on Slide 4 of the project progress report, it is highly advisable to replace any placeholder benchmarks with real empirical data derived from actual test runs to ensure that these claims rest on an evidence-backed, rigorous factual foundation.

5. Xue Haotian (薛皓天)

The system already demonstrates an impressive level of completion at this milestone. A minor recommendation regarding the evaluation framework currently under construction is to significantly scale up the volume of test data samples. Expanding the dataset size will substantially improve the statistical validity, reliability, and execution authority of the final evaluation metrics.

6. Wang Ziyu (王梓聿)

The project maintains a coherent technical flow, presenting a mature and complete pipeline extending from RAG enrichment to cohesive content integration. The decision to execute cross-model and multi-strategy comparative benchmarking experiments reflects a highly pragmatic and commendable engineering approach. The team should maintain this momentum, accelerate concluding workflows, and ensure that full-link system verification is thoroughly completed well ahead of the final course deadline.

7. Chenlei Siyu (陈雷诗语)

The codebase demonstrates excellent uniformity in coding standards and exhibits a rigorous architectural design that tightly couples with the target business scenarios. In the AI interaction layer, the decision to deploy a hybrid model combining LLM capabilities with server-side deterministic hard rules is a brilliant design choice that effectively minimizes the impact of large language model hallucinations. Moving forward into the final sprint, the team should introduce targeted unit testing and API interface testing to verify iterative stability and guarantee full-link delivery quality for the final ecosystem.

3. Action Plan for the Current Problems in the Next Milestone

The final sprint cycle runs from June 1, 2026, to June 14, 2026, allocating a total of 32 hours of focused team effort to ensure full system delivery.

3.1. System Construction Assessment & Multi-Model Benchmarking

a) Write and execute dedicated Python evaluation scripts utilizing ROUGE-L and BERTScore metrics.
b) Perform comparative analysis across three explicit generation pipelines: "Pure LLM Output", "LLM + Standard RAG", and "LLM + RAG + DeepResearch Integration" to mathematically prove data accuracy improvements.
c) Integrate the automated LLM-as-a-Judge grading pipeline to evaluate batch outputs.

3.2. Human Double-Blind Review Testing & Empirical Validation

a) Establish a standardized evaluation baseline and assemble an independent 3-to-5 person review panel.
b) Provide anonymized, randomized output samples from the three generation pipelines to the reviewers to obtain unbiased quantitative scores and annotate text hallucinations.

c) Consolidate empirical feedback into comprehensive visualization assets (radar charts and line graphs) to prove technical efficacy and compile the final Technical Verification Report.

3.3. Final System Polish, Optimization, and Prototype Deployment

a) Conduct rigorous end-to-end integration testing, repair discovered edge-case bugs, and complete full system documentation.
b) Package the application containers using Docker and execute the final prototype deployment onto the production staging environment before the hard deadline on June 14, 2026.

4. RBS

Based on the specific optimization suggestions and mandatory requirements put forward by the review experts, and in conjunction with the original module plan of the project, the core responsibility allocation table has been restructured and expanded as follows:

Module Category	Original Core Sub-tasks	Optimized Items
System Architecture & Full-Chain Stability	FastAPI full-asynchronous gateway construction Redis Stream asynchronous task processing queueSSE streaming persistent connections	**Performance Stress-Testing for Multi-Latency Paths:** Measure and profile response times across stacked workflows (FastAPI gateway, SSE streaming, and network search loops) to address latency fluctuations. **High-Concurrency Service Degradation Strategy:** Formulate concrete contingency plans to guarantee system high availability under extreme concurrent traffic loads. **Full-Chain Unit & API Interface Regression Testing:** Integrate rigorous testing protocols to verify iterative system stability across the continuous integration pipeline.
Core Business & Cost Control	Dual-path creation mode implementation Structured outline JSON Schema enforcement constraints Server-side deterministic hard rules logic engine	**API Token Consumption & Financial Auditing:** Track, benchmark, and quantify token utilization to calculate the average financial cost incurred per automated outline generation. **Deepened Multi-Model Cross-Benchmarking:** Expand

		comparative testing to systematically evaluate how different underlying models align with business logic and user intent.
Multidimensional Evaluation & Empirical Validation	Automated similarity metric scripting 3-to-5 person panel manual double-blind review sessionsSubjective and objective score fusion visualization dashboards	**Differentiated Evaluation Benchmark Metrics Design:** Separate the assessment criteria by scenario—prioritizing information extraction precision for "Report-Driven Mode" vs. requirement-gathering and context expansion efficacy for "Guided Mode." **Authentic Factual Evaluation Data Alignment:** Completely replace placeholder benchmarks on presentation sheets with actual empirical test results to ensure evidence-backed conclusions. **Scaling Test Dataset Sample Volume:** Drastically increase the volume of evaluation test cases to enhance the statistical validity, reliability, and execution authority of the metrics. **Accelerated Full-Link System Verification and Delivery:** Complete final loop testing, bug optimization, and wrap up all integration work well ahead of the course deadline to ensure seamless deployment.

5. WBS and Working Hours

The updated WBS encompasses the following core work areas, which are directly derived from the lowest-level nodes of the requirement breakdown structure and optimized based on the Milestone 2 review recommendations:

Detailed Work Breakdown Structure with Estimations (Total Budget: 220 hours)

1. Project Planning & Design (Total: 40 hrs) - [Status: Completed]				
1.1	Requirements gathering & technical feasibility analysis	All	12hrs	Completed

1.2	System architecture design (FastAPI + React + PostgreSQL)	Ma Xiaolong	10hrs	Completed
1.3	Database modeling (pgvector + Redis Stream + OSS)	Tang Chengyang	10hrs	Completed
1.4	Define 4-level PPT outline JSON Schema specification	Zhang Ruiqi	8hrs	Completed
2. Core Module Development (Total: 130 hrs) - [Status: Completed]				
2.1	**Backend Architecture Implementation**	**Ma Xiaolong**	**35hrs**	**Completed**
2.1.1	Implement FastAPI basic routing & dependency injection	Ma Xiaolong	10hrs	Completed
2.1.2	Develop Session/Task asynchronous state machine logic	Ma Xiaolong	15hrs	In Progress
2.1.3	Integrate OSS raw file storage interfaces	Ma Xiaolong	10hrs	Pending
2.2	**Prompt Engineering & Outline Generation**	**Zhang Ruiqi**	**30hrs**	**Completed**
2.2.1	Draft intent_classifier.txt intent recognition prompt	Zhang Ruiqi	8hrs	Completed
2.2.2	Optimize outline generation prompts based on JS Schema	Zhang Ruiqi	10hrs	Completed
2.2.3	API integration & comparative testing for multiple models	Zhang Ruiqi	12hrs	Completed
2.3	**RAG Retrieval Module**	**Tang Chengyang**	**35hrs**	**Completed**
2.3.1	Configure Redis Stream asynchronous processing queue	Tang Chengyang	10hrs	Completed
2.3.2	Implement vector indexing and similarity search using pgvector	Tang Chengyang	15hrs	Completed
2.3.3	Develop multi-format document parsing plugins	Tang Chengyang	10hrs	Completed
2.4	**Frontend UI Development**	**Xie Jingcheng**	**30hrs**	**Completed**
2.4.1	Set up React + TypeScript + Vite basic scaffolding	Xie Jingcheng	8hrs	Completed
2.4.2	Develop frontend SSE (Server-Sent Events) streaming client	Xie Jingcheng	12hrs	Completed
2.4.3	Build outline editing and preview interaction components	Xie Jingcheng	10hrs	Completed
3. Integration, Evaluation & Delivery (Total: 50 hrs) - [Status: Pending]				
3.1	Script end-to-end automated evaluation (ROUGE/BERTScore)	Xie Jingcheng	15hrs	Pending
3.2	DeepResearch search workflow integration	Tang Chengyang	15hrs	Pending
3.3	System integration testing & bug fixing	Ma Xiaolong	10hrs	Pending
3.4	Final prototype deployment & technical documentation delivery	All	10hrs	Pending

6. Code Quality Assessment and Optimization Actions

The team performed a stringent code review during Milestone 2 to maintain exceptional structural health and ensure compliance with modern production engineering patterns.

6.1. Code Convention & Style Consistency

The team enforces rigid formatting standards across the codebase to ensure multi-developer legibility:

- **Naming Conventions:** Strict compliance with standard Python PEP 8 stylistic parameters. Regular variables, functions, and internal modules utilize snake_case; class structures use PascalCase; global constants utilize UPPER_SNAKE_CASE.
- **Encapsulation Protection:** Internal class properties and private architectural workflows explicitly use a single leading underscore prefix '_'.

- **Layered Directory Structural Pattern:** The backend decouples concerns into 5 architectural layers: api, common, infrastructure, workers, and modules. Within each business module, file roles are uniformly divided into clear layers: controller, service, repository, and DTO.
- **Documentation Axiom:** Enforces a "Why over What" commenting paradigm. Developers must write docstrings detailing the underlying *rationale* and architectural trade-offs behind a specific implementation, rather than merely stating what the code line does.

6.2. Advanced Architectural Solutions Implemented

The team highlighted three core backend designs that directly ensure system quality, safety, and performance:

- **Adaptation to Asynchronous ORM Paradigms:** When using asynchronous database clients, relying on database-side defaults causes SQLAlchemy to mark columns as expired upon updates. Subsequent serialization of the Pydantic object triggers implicit lazy-loading, which crashes in an async environment and throws a MissingGreenlet error. Code Optimization Action: The team forced the audit timestamps inside BaseEntity to compute via Python-side callables. Values are injected directly into memory at flush time, bypassing lazy-loading entirely while maintaining database fallback defaults for independent migrations.
- **Transactional Integrity & Message Queue Synchronization:** In background worker architectures, a common critical failure is pushing an extraction message to a queue before the database transaction completes. The consumer thread picks up the message instantly, queries the DB using an independent connection, fails to find the uncommitted task record, and discards the message as invalid—rendering it a "DB orphan". The system explicitly forces database commits before queuing asynchronous tasks via Redis. Concurrently, Redis insertion is wrapped in an exponential backoff loop that retries up to 3 times. If it fails completely, the task safely remains flagged in the DB to be re-queued by automatic initialization recovery daemons.
- **Concurrency Protection & State Idempotency:** To prevent erratic state transitions from a user clicking buttons rapidly or submitting simultaneous inputs, the message gateway handles requests through a strict lock check. Every incoming request hitting handle_message queries find_running_by_session. If the session contains an active task in a PENDING, RUNNING, or STREAMING status, the system throws a BusinessException and rejects the input, ensuring session conversation history remains unpolluted.

6.3. Continuous Improvement Plan for the Final Milestone

- Maintain absolute PEP 8 compliance through pre-commit linting checks.
- Expand error boundaries within the asynchronous workers to gracefully handle third-party LLM timeouts or network cuts without dropping sessions.
- Utilize the automated verification metrics generated by the upcoming evaluation scripts to isolate edge cases where JSON schema validation catches model formatting drift, continuously tuning prompt constraints to achieve the desired 90% structural reliability

mark.

7. Reviewer's Signature